Week 1: What AI is

AI Development Course

Today

- What a language model actually does: predict the next word
- Temperature: how adventurous it is
- Tokens and embeddings: how it sees text
- Attention, and the 2017 paper behind today's AI
- Training vs. using a model
- Why models make things up
- Hands-on: the week 1 explorer, then the assignment

Probability: how likely, out of 100 tries

A forecast says **30% chance of rain**.

On 100 days like today, about 30 would be rainy.

A fair die lands on six about **1 roll in 6**.

Neither tells you what happens *this* time. Only how often, over many tries.

A language model predicts the next word

Given some text, it scores every possible next piece.

Sentence start	Model's top words
The best way to start a small business is to	have 6.4%, get 5.8%, find 5.5%
The capital of France is	Paris 31.6%, then a blank line `_____` 11.5%
Two plus two equals	four 22.7%, three 10.9%, five 6.0%

A chatbot does this over and over, one piece at a time.

Real numbers from Qwen2.5-0.5B at temperature 1.0.

Likely is not the same as true

"Two plus two equals"

- four: 22.7%
- three: 10.9%
- five: 6.0%

"three" or "five": about **17 tries in 100**, roughly 1 in 6.

Why might the model pick a likely word that's false?

Live demo: guess the next word

Explorer, section 1

1. Pick a sentence start
2. Type the word **you** think comes next
3. Press **Reveal**

Do it for 3 sentence starts. Where did your guess rank?

Temperature

The model picks at random, using the probabilities like weighted dice.

Temperature changes the weighting.

- **Low:** favourite words get even more favoured. Choices sharpen.
- **High:** differences flatten. Choices spread.

Try it: "Two plus two equals", slide from 0.2 to 2.0.

Only at 1.0 are the page's numbers the model's own. Elsewhere they are estimates.

Tokens: how the model sees text

The model chops text into pieces called **tokens**.

Text	Tokens
strawberry	3: `str` `aw` `berry`
Revenue grew 12.5% to $3,450,000 in Q3.	23 (7 words)
El cliente quiere un reembolso.	9
The best way to start a small business is to	10 (10 words)

Why tokens matter

Strawberry: "How many r's?" is hard when you never see letters, only `str` `aw` `berry`.

Cost: when your own code calls a model (weeks 9-11), you pay **per token**, for what you send and what comes back.

Numbers, rare words and some languages take more tokens for the same idea.

Live demo: tokens

Explorer, section 2

Find the most surprising token split.

A dot (`·`) marks a space at the start of a piece.

From tokens to lists of numbers

Each token is turned into a long list of numbers: an **embedding**.

Words used in similar ways end up with similar lists.

So the model can treat "refund" and "money back" alike, with no letters in common.

Week 10 builds a search tool on this: finding text by meaning, not exact words.

The 2017 paper

Attention Is All You Need

- 8 researchers, work done at Google Brain and Google Research
- Published at NIPS 2017
- Introduced the **Transformer**

The idea in one sentence: drop the word-by-word machinery and rely on attention alone.

Tested on translation: English to German, English to French. Better quality, much less time to train.

Before vs. after attention

```
BEFORE: one word at a time, passing a running summary along

  The  ->  trophy  ->  didn't  ->  fit  ->  ...  ->  it
           (each step waits for the one before it)

AFTER (models like the explorer's): each word looks back at earlier words

  The   trophy   didn't   fit   ...   it
   ^      ^        ^       ^           |
   +------+--------+-------+-----------+
          "how much does each one matter?"
```

The model in the explorer is a Transformer with 24 layers. It reads **left to right**: it looks back, never ahead.

Live demo: where the model looks

Explorer, section 3

A. Predict first. "The trophy didn't fit in the suitcase because it was too **big** / **small**."

At "it", the model hasn't seen "big" or "small" yet, so its attention at "it" is identical in both.

After you press **Reveal**:

Layer 6, one example	trophy	suitcase
big	25.4%	17.9%
small	27.9%	16.5%

"big" and "small" look back **almost the same at every layer**. Here trophy is a bit higher in both; at some other layers suitcase is slightly higher in both.

Live demo: a clear pattern, then explore

B. "Paris is in France and Berlin is in"

At layer 21, the last "in" gives **50.7%** of its attention to "Berlin". The model's next word: "Germany" (93.8%). The attention fits the right answer, but it is not proof of why.

The hidden first word here is "Paris". Include it and "in" puts **64.3%** on "Paris", 18.1% on "Berlin".

C. Pick a sentence, click a word, move the layer slider.

Numbers set the first word aside: models park a lot of attention there as a resting place, whatever the word is ("big" puts 52.8% on "The" at layer 6). Tick "Include the first word" to see it.

What attention is not

- Not a readable explanation of what the model "thinks"
- Spread across many words, not one neat arrow
- In the trophy sentences, it doesn't show which object "big" or "small" describes

Attention shows where the model looked, not proof that it understood.

Training vs. using

Training (before you meet the model)
Read huge amounts of text, predict the next token, nudge the internal numbers. The explorer's model has about half a billion of them.

Using (when you type a question)
Numbers are fixed. Predict one token, add it, predict the next.

No search engine inside. It only looks things up if given a **tool** (week 11).

Why models make things up

The job is a **likely** next word, not a **true** one.

- Usually likely and true agree
- "Two plus two equals": "three" still got 10.9%
- With no strong answer, it still writes something fluent

So check its work. **Week 12, Evaluation:** measure how often it is right on your task.

Coming up: weeks 2-14

Week	Topic	Week	Topic
2	Setting up Claude Code	9	Calling a model from code
3	Git and GitHub	10	Embeddings and RAG
4	Planning a project	11	Tools, agents and MCP
5	Building with AI	12	Evaluation
6	Reviewing code	13	Security and responsible use
7	Running what you build	14	Final demos
8	Debugging, midterm check-in		

This week's assignment

1. List **3 problems** from your work, studies or life where AI might help. Tag each **good fit** or **poor fit** for a next-word predictor, with one sentence why.
2. One paragraph on the **most surprising thing** you saw in the explorer, quoting a number from it.

Hand in one document, about one page. 10 points.

Explorer: vasuchettyphd.github.io/ai-dev-course/weeks/01-what-is-ai/explorer/